

Automation with AI and MCP Servers using Playwright-Notes-5

- **Building Playwright Test Framework-**
- **Automation Testing Framework-** structured approach or set of guidelines/rules for designing, creating and executing automated test cases effectively-serves as a foundation for writing test scripts, managing test data and organizing test execution, ensuring reusability, maintainability, scalability and consistency in the automation process

Automation Testing Framework — What Is It?

A testing framework is a **pre-built toolkit that helps you write, organize, and run automated tests** — so you don't have to build everything from scratch.

Think of it like a **kitchen with all the appliances already installed**. You bring the ingredients (your tests), the kitchen (framework) provides the tools, structure, and workflow to cook with.

Why Not Just Write Tests Without One?

You *could* — but you'd end up manually building:

- How tests are organized
- How results are reported
- How failures are caught
- How tests are run together
- How to set up and clean up before/after tests

Frameworks give you all of that **out of the box**.

The Big Categories

1. Unit Testing Frameworks

Test **tiny individual pieces** of code in isolation — one function, one method.

js

```
// Does this function add numbers correctly?
```

```
test('adds 2 + 2', () => {  
  expect(add(2, 2)).toBe(4);  
});
```

Language	Popular Frameworks
JavaScript	Jest, Mocha, Vitest
Python	PyTest, unittest
Java	JUnit, TestNG
C#	NUnit, xUnit

Like testing each **individual ingredient** before cooking the meal.

2. End-to-End (E2E) Testing Frameworks

Test the **entire application** from a real user's perspective — click buttons, fill forms, navigate pages.

```
js
```

```
// Does the whole login flow work?
```

```
await page.goto('/login');  
await page.fill('#email', 'user@test.com');  
await page.fill('#password', 'secret');  
await page.click('#submit');  
await expect(page).toHaveURL('/dashboard');
```

Framework	Language	Notes
Playwright	JS/Python/Java/C#	Modern, fast, reliable
Cypress	JavaScript	Great for web apps
Selenium	Many languages	The OG, widely used
Puppeteer	JavaScript	Chrome-focused

Like hiring a **mystery shopper** to walk through your entire store and report what's broken.

3. API Testing Frameworks

Test your **backend APIs** — send requests, check responses, no browser needed.

```
js
// Does the API return the right data?
const response = await request.get('/api/users/1');
expect(response.status()).toBe(200);
expect(response.body.name).toBe('John');
```

Framework	Notes
Postman Newman	/ GUI + command line
REST Assured	Java-based
Supertest	Node.js
Playwright	Can do API testing too

Like testing a **restaurant's kitchen orders** directly — without ever going through the dining room.

4. Visual Testing Frameworks

Check that the UI **looks correct** — catches pixel-level design regressions.

```
js
// Does the homepage still look the same?
expect(await page.screenshot()).toMatchSnapshot();
// FAILS if even one pixel changed unexpectedly
```

Framework	Notes
Percy	Cloud-based visual testing
Applitools	AI-powered visual comparison

Playwright Snapshots	Built-in comparison	screenshot
---------------------------------	------------------------	------------

Like taking a **photo of your product** and comparing it to yesterday's photo — spot any differences instantly.

5. Performance Testing Frameworks

Test how fast and stable your app is under **heavy load**.

Framework	Notes
k6	Modern, developer-friendly
JMeter	Java-based, widely used
Locust	Python-based

Like seeing what happens when **10,000 people show up to your store at once**.

How a Framework Is Structured

Testing Framework

—	Test Runner ← Finds and runs all your tests
—	Assertion Library ← Checks if results are correct
—	Reporting ← Shows pass/fail results clearly
—	Setup / Teardown ← Prepares and cleans up before/after tests
—	Mocking / Stubbing ← Fakes external services for isolated tests

Core Concepts Every Framework Has

Test Suite — A group of related tests

```
js
describe('Login Page', () => { // ← Suite
  test('shows error on wrong password', ...);
  test('redirects on success', ...);
});
```

Test Case — One specific thing being tested

```
js
test('button turns green when clicked', () => {
  // ...
});
```

Assertions — The "did it work?" check

```
js
expect(result).toBe(true);    // passes if true
expect(list).toHaveLength(3); // passes if 3 items
expect(page).toHaveURL('/home'); // passes if on home page
```

Setup & Teardown — Runs before/after tests

```
js
beforeEach(() => { /* set up fresh state */ });
afterEach(() => { /* clean up after each test */ });
beforeAll(() => { /* run once before all tests */ });
afterAll(() => { /* run once after all tests */ });
```

The Testing Pyramid

```

  ^
 / \      ← E2E Tests (few, slow, expensive)
/ E2E \    "Does the whole app work?"
/-----\
 /      \ ← Integration Tests (some)
/ Integration \ "Do the pieces work together?"
/-----\
 /          \ ← Unit Tests (many, fast, cheap)
/ Unit Tests  \ "Does each function work?"
/ _____ \
```

Rule of thumb: Lots of unit tests, some integration tests, a few E2E tests.

Popular Full Framework Stacks

Stack	Use Case
Playwright + Jest	Web E2E + unit testing
Cypress + Mocha	Frontend web apps

Selenium + TestNG	Enterprise Java apps
PyTest + Playwright	Python-based web testing
JUnit + REST Assured	Java API testing

The Simple Mental Model

A framework is like a **sports league** for your tests:

- The **league** (framework) provides the rules, scoreboard, and schedule
- The **teams** (test suites) are groups of related tests
- The **players** (individual tests) do the actual work
- The **referee** (assertions) decides pass or fail
- The **scoreboard** (reporter) shows you the final results

Without the league, you'd just have people randomly kicking a ball around with no structure.

- **Framework Development Guidelines-**
 1. Use **Playwright MCP server** code generation to generate initial code
 2. Convert generated code into a **Typescript Playwright Test Runner framework**
 3. Implement **Page Object Model (POM)** for all pages
 4. Use **fixtures with POM**. Also use **fixtures** instead of 'beforeEach' and 'afterEach' hooks wherever appropriate
 5. Store **test data** in a JSON file and read dynamically
 6. Keep tests **independent**-no inter-test dependencies
 7. **Assertions** must be in test scripts-not in POM methods
 8. Configure **playwright.config.ts** to include:
 - HTML reports
 - 1 retry for failed tests
 - Video recording, screenshots and trace
 9. **Add tags and annotations** for each test:
 - Include **Type** (Eg: Smoke, Regression, Sanity)
 - Include **Description** explaining the test purpose
 10. **Assertions** must be in test scripts-not in POM methods Create a **.gitignore file to ignore result folders and node_modules**
 11. Implement **CI pipelines using Github Actions**

12. Follow **proper naming conventions** as per coding standards

In VS Code:

Refer to the Playwright.md file and type the following prompt:

Follow the instructions provided in the Playwright_Framework_Prompt.m file. Follow each instruction given in the .md file step by step like Step 1>Step 2>Step 3. Use Playwright MCP server to perform browser actions and to generate code using codegen.

- **Playwright Test Agents-** Playwright comes with 3 Playwright agents: **Planner, Generator and Healer**-These agents can be used independently, sequentially or as the chained calls in the agentic loop-Using them sequentially will produce test coverage for your product
- **Command:** `npx playwright init-agents --loop=vscode` (first install playwright using `npx playwright install`)
- **Planner-** explores your app and produces a test plan for one or many scenarios and user flows-produces a markdown test plan (.md file)
- **Planner input:** A clear request to the planner (e.g: "Generate a plan for Login functionality")-a seed test that sets up the environment necessary to interact with your app-A Product Requirement Document (PRD) for context
- **Corresponding Planner output:** A markdown test plan saved as specs/basic-operations.md-the plan is human-readable but precise enough for test generation
- **Prompt-** **Generate a test plan for <https://www.saucedemo.com/> login functionality using Playwright agents planner**
- **Generator-** uses the markdown plan to produce executable Playwright Tests-verifies selectors and assertions live as it performs the scenarios
- **Generator Input-** Markdown plan-we can provide markdown test plan in the context of generator-that is we upload the file which the planner has generated along with our prompt-this is how the generator knows where to get the test plan from-alternatively you can mention the file name in the prompt
- **Corresponding Generator output-** A test suite under tests/ folder-generated tests may include initial errors that can be healed automatically by the healer agent
- **Prompt-** **Generate a playwright test script for: 1. Standard user login (Happy path) and 2. Locked out user**
- **Healer-** executes the test suite and automatically repairs failing tests-replays failing test steps-then inspects the current UI to locate equivalent elements or flows-after that it suggests a patch or remedy to fix the failing test/step such as a locator update, wait adjustment, data fix, etc-and then it reruns the test until it passes or until guardrails stop the loop
- **Healer Input-** Failing test name

- **Corresponding Healer Output-** A passing test or a skipped test if the healer believes that the functionality is broken
- **Prompt- Fix failed tests standard-user-login.spec.ts and locked-out-user-login.spec.ts**

A **PRD** (Product Requirements Document) is a document that describes what a product or feature should do — it serves as the blueprint for building something.

It typically covers:

- **Purpose** — why the product/feature exists and what problem it solves
- **Goals & success metrics** — how you'll know it worked
- **User stories / use cases** — who uses it and how
- **Functional requirements** — what the product must do
- **Non-functional requirements** — performance, security, scalability constraints
- **Out of scope** — what's explicitly *not* being built
- **Timeline & milestones** — rough delivery expectations

It's the primary communication tool between product managers, engineers, designers, and stakeholders — ensuring everyone builds the same thing for the same reasons.

Example of a PRD:

PRD: Saved Searches Feature

Product: JobFinder Mobile App

Author: Jane Smith, Product Manager

Status: Draft

Last Updated: May 12, 2026

1. Overview

Users frequently repeat the same job searches (e.g., "remote React developer in Chicago"). Today, they must re-enter filters every session, which is tedious and causes drop-off. This feature lets users save and re-run searches with one tap.

2. Goals

- Reduce time-to-search for returning users by 60%
 - Increase weekly active users by 10% within 90 days of launch
 - Decrease search-related support tickets by 20%
-

3. User Stories

- **As a job seeker**, I want to save my search criteria so I don't have to re-enter them each time I open the app.
 - **As a job seeker**, I want to be notified when new jobs match a saved search, so I can apply quickly.
 - **As a user**, I want to manage (rename, delete) my saved searches so my list stays organized.
-

4. Functional Requirements

4.1 Saving a Search

- A "Save Search" button appears on the search results screen.
- Tapping it prompts the user to name the search (default: auto-generated name from filters, e.g., "Remote React – Chicago").
- Saved searches store: keywords, location, job type, salary range, date posted filter, and remote/on-site preference.
- Maximum of 10 saved searches per user (free tier); 50 for premium users.

4.2 Accessing Saved Searches

- A "Saved Searches" tab appears in the main navigation.
- Each saved search card displays: name, filter summary, number of new jobs since last viewed, and last run date.
- Tapping a card re-runs the search with the saved filters applied.

4.3 Notifications

- Users can toggle push notifications per saved search.
- Notification triggers when ≥ 1 new job matches a saved search (checked every 6 hours).
- Notification copy: "5 new jobs match your 'Remote React – Chicago' search."

4.4 Managing Saved Searches

- Users can rename or delete any saved search by long-pressing the card.
 - Deleting a saved search also disables its notifications.
-

5. Non-Functional Requirements

- Search re-run must return results in under 2 seconds (p95).
 - Saved search data must sync across devices within 30 seconds.
 - Feature must work on iOS 16+ and Android 12+.
 - Saved searches must be stored server-side (not local-only) to support cross-device sync.
-

6. Out of Scope

- Sharing saved searches with other users
 - Collaborative job searching
 - AI-suggested saved searches (future consideration)
-

7. Success Metrics

Metric	Baseline	Target (90 days)
Avg. time to first search result	45s	18 sec

	e	
	c	
Weekly active users	132,000	
	0	
	0	
	0	
	0	
	0	
Saved search adoption rate	0%	25% of active users

8. Timeline

Milestone	Date
Design mockups complete	May 26, 2026
Engineering kickoff	June 2, 2026
Internal QA	June 23, 2026
Beta release	June 30, 2026
Public launch	July 14, 2026

9. Open Questions

1. Should we allow saved searches to be reordered (drag-and-drop)?

2. What happens to saved searches if a user downgrades from premium to free (over the 10-search limit)?
3. Do notifications need a quiet-hours setting?

Playwright Test Agents

Think of it like a software testing factory with three workers, each with a specific job:

The Planner

"The one who figures out *what* to test"

Before writing a single line of code, the Planner looks at your app and asks:

- *What are the important things users do here?*
- *What screens and buttons exist?*
- *What could go wrong?*

It then produces a testing checklist — a structured list of scenarios like "*user logs in*", "*user adds item to cart*", "*user checks out*". It's the strategist that turns a vague goal like "*test my website*" into a concrete plan.

The Generator

"The one who *writes* the actual test code"

Once there's a plan, the Generator takes each scenario and writes the Playwright test script for it — the actual code that clicks buttons, fills forms, and checks results.

It handles the boring, repetitive part: "*Click this selector, type this text, assert this element is visible.*" You'd normally spend hours writing this by hand — the Generator does it in seconds.

The Healer

"The one who *fixes* broken tests"

Tests break constantly — not because your app is broken, but because a button got renamed, a CSS class changed, or a page was restructured. The Healer watches for these failures and automatically updates the test to match the new reality.

Without a Healer, you'd manually hunt down every broken selector yourself. The Healer is basically self-maintaining tests.

How they work together

Your App



PLANNER → "Here are 12 things we should test"



GENERATOR → "Here is the Playwright code for all 12"



HEALER → "Test 4 broke after the UI update — fixed it"

The goal is a fully automated testing pipeline where you describe what you want tested, and the agents handle the rest — writing, running, and maintaining the tests on your behalf.